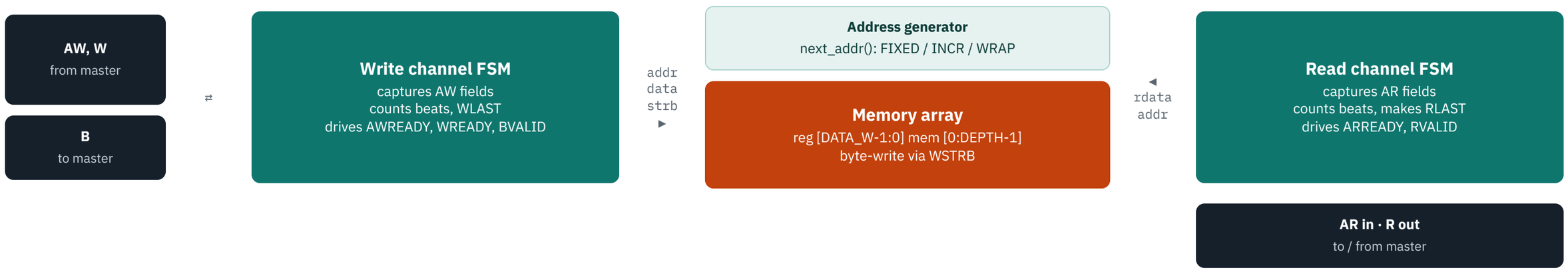
DV INTERVIEW · GLANCE SHEET 8 · PROJECT PART 1

Building an AXI Slave RAM in Verilog

A memory that speaks AXI: two independent state machines, an address generator, and byte-enabled writes.

`Write FSM: IDLE → DATA → RESP`
`Read FSM: IDLE → DATA`
`mem index = addr >> log2(DATA_W/8)`

1 Architecture

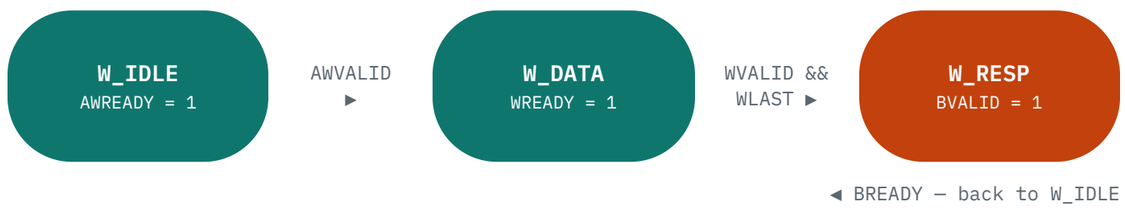


Read and write FSMs are independent → a read and a write can be in progress at the same time, just as AXI intends.

Address generator is shared logic — the same function is used by both FSMs, so burst rules are coded once.

Parameters: `ADDR_W`, `DATA_W`, `ID_W`, `DEPTH`. STRB width = DATA_W/8.

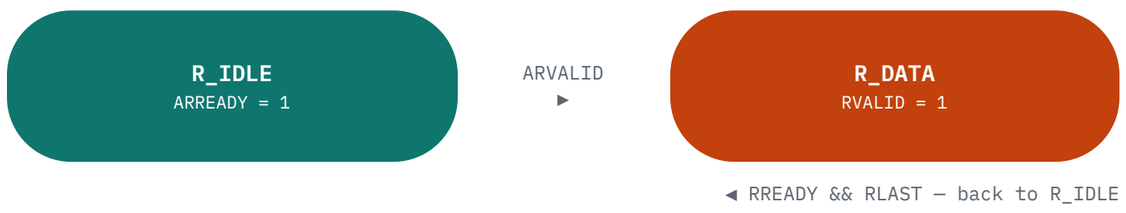
2 Write FSM



State	Does	Leaves when
W_IDLE	AWREADY HIGH; on handshake latch AWID, AWADDR, AWLEN, AWSIZE, AWBURST	AWVALID && AWREADY
W_DATA	WREADY HIGH; each WVALID beat writes memory with WSTRB, advances address	WVALID && WREADY && WLAST
W_RESP	BVALID HIGH with BID = latched AWID, BRESP = OKAY / SLVERR	BVALID && BREADY

This slave **waits for AW before accepting W**. That's legal (a slave may wait for AWVALID before WREADY) and keeps the design simple, but your **master/TB must not wait for WREADY before AWVALID**.

3 Read FSM



State	Does	Leaves when
R_IDLE	ARREADY HIGH; latch ARID, ARADDR, ARLEN, ARSIZE, ARBURST; clear beat counter	ARVALID && ARREADY
R_DATA	RVALID HIGH; RDATA = mem[addr]; RLAST = (count == ARLEN); on each handshake advance address & count	RVALID && RREADY && RLAST

RDATA stays stable while RREADY is LOW because the address only moves on a handshake — the handshake rule is met for free.

With a real **synchronous** block RAM, data appears a cycle after the address. Then you need a pre-fetch stage or a small output register/skid buffer so RVALID and RDATA line up.

4 Key Verilog pieces

WRITE FSM (CORE)

```
localparam W_IDLE = 2'd0, W_DATA = 2'd1, W_RESP = 2'd2;

always @(posedge ACLK or negedge ARESETn)
  if (!ARESETn) wstate <= W_IDLE;
  else case (wstate)
    W_IDLE: if (AWVALID) begin            // AWREADY = 1 here
              aw_id   <= AWID;    w_addr  <= AWADDR;
              aw_len  <= AWLEN;   aw_size <= AWSIZE;
              aw_burst<= AWBURST; wstate  <= W_DATA;
            end
    W_DATA: if (WVALID) begin             // WREADY = 1 here
              w_addr <= next_addr(w_addr, aw_len, aw_size, aw_burst);
              if (WLAST) wstate <= W_RESP;
            end
    W_RESP: if (BREADY) wstate <= W_IDLE; // BVALID = 1 here
  endcase

assign AWREADY = (wstate == W_IDLE);
assign WREADY  = (wstate == W_DATA);
assign BVALID  = (wstate == W_RESP);
assign BID     = aw_id;
assign BRESP   = 2'b00;                   // OKAY
```

BYTE-ENABLED MEMORY WRITE

```
integer i;
always @(posedge ACLK)
  if (WVALID && WREADY)
    for (i = 0; i < DATA_W/8; i = i + 1)
      if (WSTRB[i])
        mem[w_addr >> ADDR_LSB][8*i +: 8] <= WDATA[8*i +: 8];
// ADDR_LSB = log2(DATA_W/8), e.g. 2 for 32-bit
```

READ FSM (CORE)

```
localparam R_IDLE = 1'b0, R_DATA = 1'b1;

always @(posedge ACLK or negedge ARESETn)
  if (!ARESETn) rstate <= R_IDLE;
  else case (rstate)
    R_IDLE: if (ARVALID) begin            // ARREADY = 1 here
              ar_id <= ARID;   r_addr  <= ARADDR;
              ar_len<= ARLEN;  ar_size <= ARSIZE;
              ar_burst <= ARBURST; r_cnt <= 0;
              rstate <= R_DATA;
            end
    R_DATA: if (RREADY) begin             // RVALID = 1 here
              r_addr <= next_addr(r_addr, ar_len, ar_size, ar_burst);
              r_cnt  <= r_cnt + 1;
              if (RLAST) rstate <= R_IDLE;
            end
  endcase

assign ARREADY = (rstate == R_IDLE);
assign RVALID  = (rstate == R_DATA);
assign RDATA   = mem[r_addr >> ADDR_LSB];
assign RLAST   = (r_cnt == ar_len);
assign RID     = ar_id;
assign RRESP   = 2'b00;
```

NEXT-ADDRESS FUNCTION (ALL BURST TYPES)

```
function [ADDR_W-1:0] next_addr(input [ADDR_W-1:0] a,
    input [7:0] len, input [2:0] size, input [1:0] burst);
  reg [ADDR_W-1:0] nb, win, inc;
  begin
    nb  = 1 << size;
    inc = (a & ~(nb - 1)) + nb;          // align, then step
    win = nb * (len + 1);
    case (burst)
      2'b00:   next_addr = a;                              // FIXED
      2'b10:   next_addr = (a & ~(win-1)) | (inc & (win-1)); // WRAP
      default: next_addr = inc;                            // INCR
    endcase
  end
endfunction
```

5 Design choices

Choice	Effect
One write + one read in flight	Simple; no ID queues; always in-order
Separate R / W FSMs	Reads and writes run concurrently
Wait for AW before W	Legal; no W buffering needed
Align address after beat 1	Handles unaligned starts correctly
Word-indexed memory + WSTRB	Narrow and unaligned writes work
Out-of-range address	Return SLVERR (still finish all beats)
BURST = 2'b11	Treat as error → SLVERR

Upgrade path: add AW/AR FIFOs for multiple outstanding, then per-ID tracking to allow out-of-order.

! Classic bugs in AXI slaves

- WREADY HIGH before AW is latched → data written to the wrong address.
- RLAST off by one (counter compared to LEN+1 instead of LEN).
- BVALID dropped before BREADY → response lost.
- Beat-2 address = start + size instead of **aligned** start + size.
- Ignoring WSTRB → narrow writes clobber neighbour bytes.
- VALIDs not forced LOW in reset.
- Ending the burst on count but ignoring WLAST mismatch (should flag).
- Using AW fields directly instead of latched copies after handshake.
- RDATA changing while RVALID HIGH and RREADY LOW.

? Test plan & Q&A

DIRECTED TESTS FIRST

- Single write → read back.
- INCR / WRAP / FIXED burst write, then read back.
- Partial WSTRB, narrow and unaligned writes.
- BREADY / RREADY held LOW for N cycles.
- Read and write at the same time.
- Reset in the middle of a burst.

Question	Answer
Why two FSMs?	R and W channels are independent.
Why latch AW fields?	Master may change AW after the handshake.
How is WSTRB used?	Per-byte write enable on the memory word.
Is this slave in-order?	Yes — one transaction per direction at a time.